QuickSort

1. High-level idea

QuickSort sorts by **partitioning**: pick a pivot, reorder the array so smaller elements go left and larger go right, then recursively sort the two parts.

It's a classic divide-and-conquer algorithm that is usually **in-place** and very fast in practice.

---

2. Algorithm skeleton (pseudocode)

```
QUICKSORT(A, low, high)
    if low < high
        p = PARTITION(A, low, high)
        QUICKSORT(A, low, p-1)
        QUICKSORT(A, p+1, high)
```

Different `PARTITION` implementations return different pivot indices and require slightly different recursive bounds.

---

3. Partition schemes (short summary)

- **Lomuto partition** (simple): pivot = A[high]; uses index `i` that ends up at pivot final position. Tends to do more swaps.
- **Hoare partition** (classic): pivot = A[mid] or A[low]; uses two indices `i` and `j` that move inward and swap. Fewer swaps, different recursive bounds.
- **3-way (Dutch National Flag)**: partitions into `< pivot`, `== pivot`, `> pivot` — great when many duplicates.
- **First-element pivot**: pivot = A[low] — simple but *bad* on already-sorted inputs (causes worst-case $O(n^2)$) unless randomized.

---

4. Complexity

- **Best case:** $\Theta(n \log n)$ — balanced splits
- **Average (randomized pivot):** $\Theta(n \log n)$
- **Worst case:** $\Theta(n^2)$ — e.g., always pick smallest/largest pivot
- **Space:** $O(\log n)$ expected recursion stack; worst-case $O(n)$
- **Stability:** Not stable by default
- **In-place:** Yes (typical implementations)

---

5. Correctness sketch

1. Partition places pivot into final sorted position (invariant: elements left ≤ pivot and right ≥ pivot, depending on scheme).
2. Recursively sorting left and right subarrays yields fully sorted array.
3. Prove by induction on array length.

Use loop invariants to argue partition correctness.

---

6. Practical optimizations

- **Randomized pivot** — swap a random element with pivot before partition to avoid adversarial inputs.
- **Median-of-three pivot** — pick median of `A[low], A[mid], A[high]`.
- **Cutoff to insertion sort** — for small subarrays (e.g., ≤ 10), insertion sort is faster.
- **Tail recursion elimination** — always recurse on smaller partition first and loop on larger to guarantee $O(\log n)$ stack.
- **3-way partitioning** — improves performance when many equal keys.
- **Hybrid sorts** — intro sort (std::sort) switches between quick/heap/insert to get worst-case guarantees.

---

7. When to use QuickSort vs MergeSort (short)

- Use **QuickSort** when you want an in-place, typically fastest practical sort.
- Use **MergeSort** if stability or worst-case Θ(n log n) is required (or for external sorting).
- Many standard libraries use hybrid strategies (e.g., introsort).

---

8. Common pitfalls

- Off-by-one errors in partition loops.
- Misinterpreting Hoare's returned index and using wrong recursion bounds.
- Using first-element pivot on sorted input → $\Theta(n^2)$. Teach randomized/median fixes.
- Forgetting to stop recursion for small subarrays or not applying insertion sort cutoff.

---

9. Code examples (clean, standard C++)

A — First-element pivot partition (pivot = A[low]) — `i` advances when `A[i] <= pivot`

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// Partition using first element as pivot; i advances while A
[i] <= pivot
int partitionFirstPivot(vector<int>& A, int low, int high) {
    int pivot = A[low];
```

```
    int i = low + 1;
    int j = high;

    while (true) {
        while (i <= high && A[i] <= pivot) ++i;
        while (j >= low + 1 && A[j] > pivot) --j;
        if (i < j) swap(A[i], A[j]);
        else break;
    }
    swap(A[low], A[j]); // put pivot into final position
    return j;
}

void quickSortFirstPivot(vector<int>& A, int low, int high) {
    if (low < high) {
        int p = partitionFirstPivot(A, low, high);
        quickSortFirstPivot(A, low, p - 1);
        quickSortFirstPivot(A, p + 1, high);
    }
}

// driver for demo
int main() {
    vector<int> A = {8, 3, 5, 4, 7, 6, 1, 2};
    quickSortFirstPivot(A, 0, (int)A.size() - 1);
    for (int x : A) cout << x << ' ';
    cout << '\\n';
    return 0;
}
```

B — Lomuto partition with randomized pivot (simple & robust)

```
#include <iostream>
#include <vector>
#include <random>
```

```
#include <algorithm>
using namespace std;

int lomutoPartition(vector<int>& A, int low, int high) {
    int pivot = A[high];
    int i = low - 1;
    for (int j = low; j <= high - 1; ++j) {
        if (A[j] <= pivot) {
            ++i;
            swap(A[i], A[j]);
        }
    }
    swap(A[i + 1], A[high]);
    return i + 1;
}

void randomizedQuickSort(vector<int>& A, int low, int high, m
t19937 &rng) {
    if (low < high) {
        uniform_int_distribution<int> dist(low, high);
        int r = dist(rng);
        swap(A[r], A[high]); // random pivot to end
        int p = lomutoPartition(A, low, high);
        randomizedQuickSort(A, low, p - 1, rng);
        randomizedQuickSort(A, p + 1, high, rng);
    }
}

int main() {
    vector<int> A = {8,3,5,4,7,6,1,2};
    mt19937 rng(chrono::steady_clock::now().time_since_epoch
().count());
    randomizedQuickSort(A, 0, (int)A.size() - 1, rng);
    for (int x : A) cout << x << ' ';
    cout << '\\n';
```

```
    return 0;
}
```

C — 3-way (Dutch National Flag) QuickSort (good for many duplicates)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void threeWayQuickSort(vector<int>& A, int low, int high) {
    if (low >= high) return;
    int pivot = A[high]; // or randomize
    int lt = low, i = low, gt = high;
    while (i <= gt) {
        if (A[i] < pivot) swap(A[lt++], A[i++]);
        else if (A[i] > pivot) swap(A[i], A[gt--]);
        else ++i;
    }
    threeWayQuickSort(A, low, lt - 1);
    threeWayQuickSort(A, gt + 1, high);
}
```